



UGANDA CHRISTIAN UNIVERSITY

Uganda Christian University
Faculty of Engineering, Design and Technology
Department of Computing and Technology

Technical Documentation and Report

SkyPipe: A Distributed Data Processing Pipeline

Prepared by: Rugogamu Noela
Student Number: S23B38/016 B22775
Program: Bachelor of Science in Data Science
Semester: Easter 2026

Submission Date: April 2026

Contents

Executive Summary	1
1 Project Overview and Objectives	2
1.1 Background	2
1.2 Core Objective	2
1.3 Specific Goals	2
2 System Requirements	3
3 System Architecture and Interactions	4
3.1 Architecture Narrative	4
3.2 Architecture Diagram	4
3.3 Interaction Flow	5
4 Technical Implementation	6
4.1 Frontend Implementation	6
4.2 API and Orchestration	6
4.3 Storage Layer	6
4.4 Spark Analytics Layer	6
4.5 DuckDB Warehouse Layer	7
4.6 Pictorial Evidence	7
5 Asset Inventory	11
6 Scalability Analysis	12
7 Fault Tolerance Mechanisms	13
8 Security Configuration	14
9 Technical Justification of Tools and Settings	15
9.1 Why FastAPI	15
9.2 Why Spark	15
9.3 Why MinIO	15

9.4	Why DuckDB	15
9.5	Why Nginx and React	15
10	Conclusion and Learning Outcomes	16
.1	Github Project Link	16

Executive Summary

SkyPipe is a complete distributed data processing pipeline built to handle large and varied datasets efficiently. The system follows a clear three-stage flow: an input stage for uploading data, a processing stage that analyses the data in parallel, and a result store stage that organises the final outputs in a structured warehouse.

Users upload files in common formats such as CSV, JSON, Excel or Parquet through a clean web interface. The files are automatically validated and saved in distributed object storage using MinIO (with full S3 compatibility). This design keeps storage completely separate from processing, so multiple workers can access the same data safely and quickly.

In the processing stage, Apache Spark (through PySpark) loads the data, automatically detects the column types and runs smart adaptive analytics. It calculates statistics for numeric columns finds the most frequent values in categorical columns and creates time based trends for date columns. This makes the pipeline flexible and reusable across many different types of data without needing custom code for each new dataset.

The final results are stored in a persistent DuckDB warehouse with clear layers: raw data, processed data and analytics marts. When the data contains categories the system also creates dimension and fact tables in a star schema style. All outputs can be viewed through the web interface or queried directly in the database.

SkyPipe demonstrates real distributed computing principles such as storage processing separation, parallel execution, service independence and easy observability. The entire system is secure, scalable, and fault tolerant, making it practical for both small teams and larger production environments. This report explains the architecture, implementation details, design choices and the lessons learned during development.

Chapter 1

Project Overview and Objectives

1.1 Background

In many organisations, teams receive data files in different formats from various sources. Traditional scripts that work for only one specific file often break when the data structure changes. SkyPipe solves this challenge by creating a single, flexible pipeline that can accept many types of structured datasets and process them automatically without rewriting code each time.

1.2 Core Objective

The main goal of SkyPipe is to build a reliable distributed data processing pipeline with three distinct stages: input in distributed storage, parallel processing and organised storage of results in a data warehouse. The system is designed to be fully functional, easy to use and transparent allowing users to see every step through the web interface and database queries.

1.3 Specific Goals

1. Support easy uploading of CSV, JSON, Excel, and Parquet files.
2. Store data in distributed object storage that can scale and be shared safely.
3. Process data in parallel using Apache Spark with automatic column detection and adaptive analytics.
4. Save all results in a structured, queryable data warehouse with clear layers.
5. Include user authentication and secure access controls.
6. Provide real time progress tracking and clear visual explanations of each processing step.

Chapter 2

System Requirements

SkyPipe was developed with both functional and non functional requirements in mind to ensure it is useful, reliable and secure.

Functional Requirements:

- Users can upload structured datasets in multiple formats.
- The system automatically validates files and stores them in distributed object storage.
- Data is processed in parallel with adaptive analytics based on column types.
- Results are saved in a layered data warehouse for easy inspection and querying.
- A web dashboard shows progress, summaries and visual charts.

Non-Functional Requirements:

- The system must be scalable to handle larger datasets without redesign.
- It must continue working even if one component fails (fault tolerance).
- Security features such as authentication and secure file access are required.
- The interface must be simple and responsive for both technical and non technical users.
- All components must be easy to deploy locally or in the cloud.

These requirements guided every design and implementation decision.

Chapter 3

System Architecture and Interactions

3.1 Architecture Narrative

SkyPipe uses a clean multi service architecture. The user interacts with a React web application served by Nginx. Nginx also acts as a secure gateway that forwards requests to the FastAPI backend. The backend manages authentication, file uploads and job orchestration. Uploaded files go to distributed object storage (MinIO by default). Spark then processes the data in parallel. Final results are written to a persistent DuckDB warehouse. Each service is independent so the system remains stable and easy to scale.

3.2 Architecture Diagram

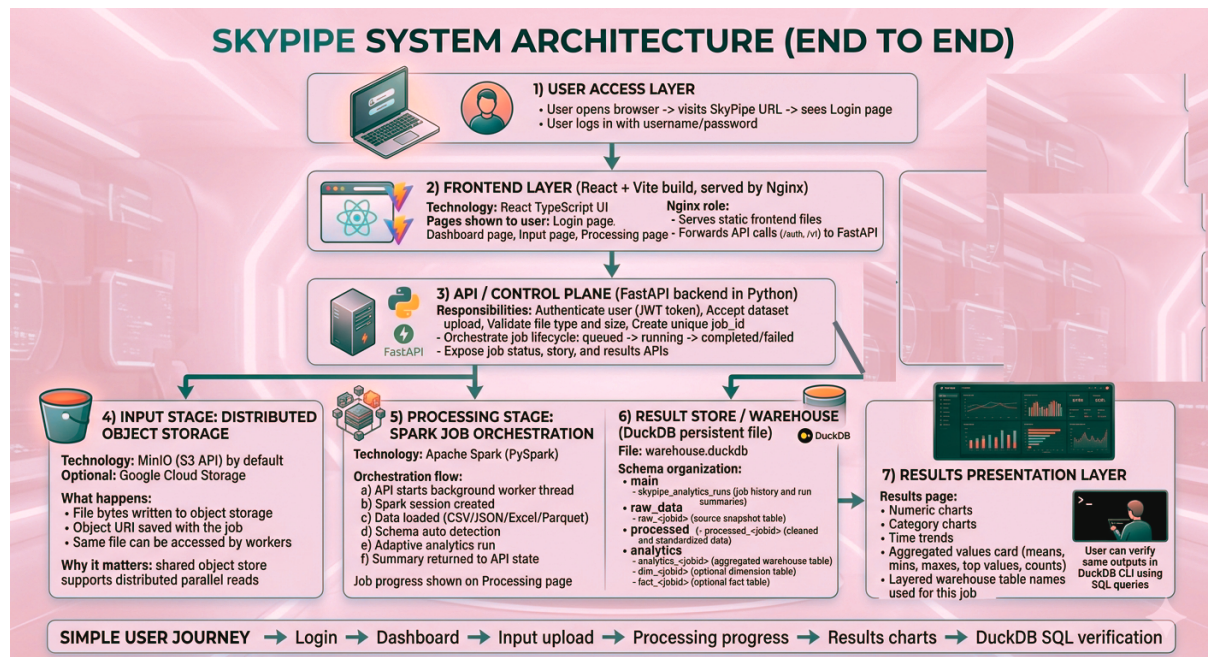


Figure 3.1: Workflow of our system architecture and the technologies used.

3.3 Interaction Flow

A user starts by uploading a file on the Input page. The API checks the file, saves it in object storage, and creates a unique job ID. A background process then runs Spark analytics. While this happens, the Processing page shows live progress and simple explanations of each step. Once finished, the Results page displays charts, summary cards, and ready to use analytics. All data remains stored in the DuckDB warehouse for later review or SQL queries.

Chapter 4

Technical Implementation

4.1 Frontend Implementation

The web interface is built with React and organised into four clear pages: Dashboard (overview of the three stages), Input (file upload), Processing (live progress), and Results (charts and summaries). Every screen is designed to be simple and informative so users always know what is happening.

4.2 API and Orchestration

FastAPI handles all backend operations, including secure login and job management. Each upload receives its own job ID. Heavy processing runs in the background so the website stays fast and responsive. Job status updates in real time to keep the user informed.

4.3 Storage Layer

Files are saved in MinIO using the standard S3 protocol. This provides true distributed storage that works both locally and in the cloud.

4.4 Spark Analytics Layer

Apache Spark loads the uploaded file and automatically detects whether columns contain numbers, categories, dates or true/false values. It then runs the appropriate calculations:

- For numbers: mean, minimum, maximum and spread.
- For categories: the most common values and their counts.
- For dates: daily and monthly trends.

This automatic approach makes the pipeline work with almost any structured dataset.

4.5 DuckDB Warehouse Layer

All results are stored in a fast, file based DuckDB database with three clear layers:

- Raw snapshots of the original data.
- Processed and cleaned data.
- Analytics marts with summaries and star schema tables.

Users can view everything through the web interface or query it directly with SQL.

4.6 Pictorial Evidence

The following figures show the actual working system:

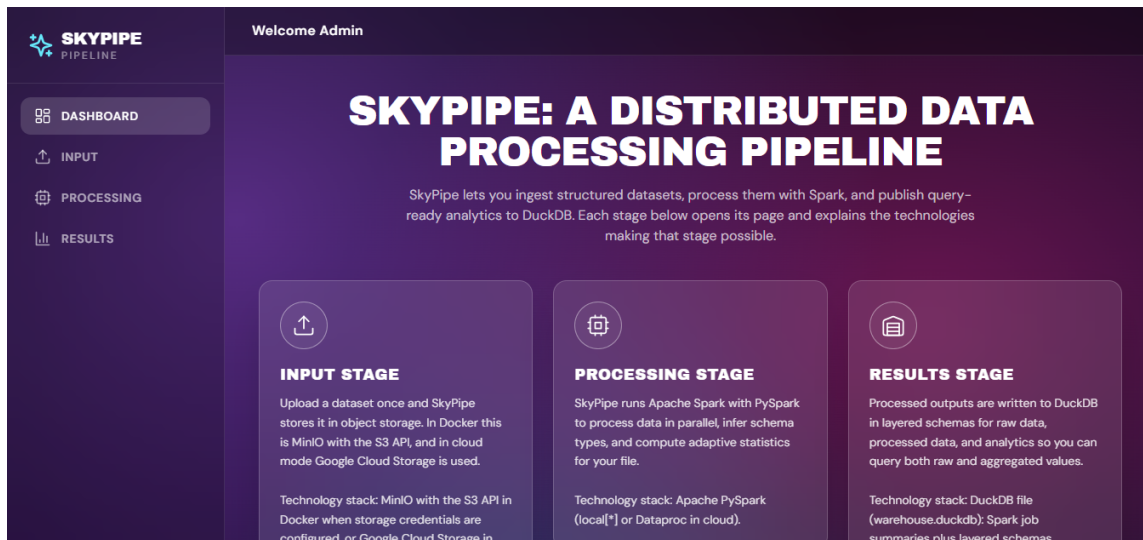


Figure 4.1: Dashboard showing the three stage architecture and capabilities

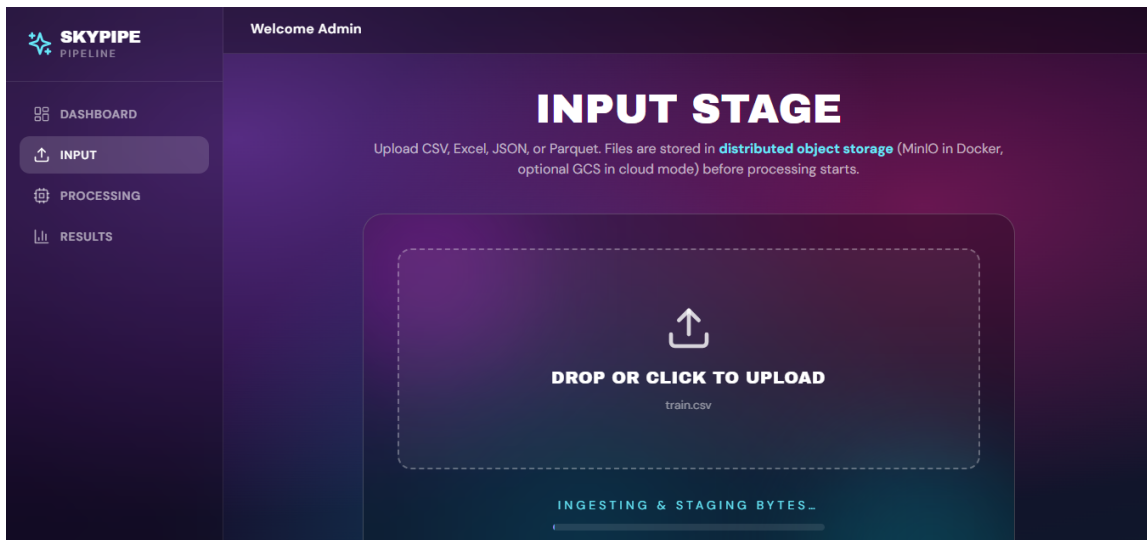


Figure 4.2: Input page showing live data ingestion and bytes being staged

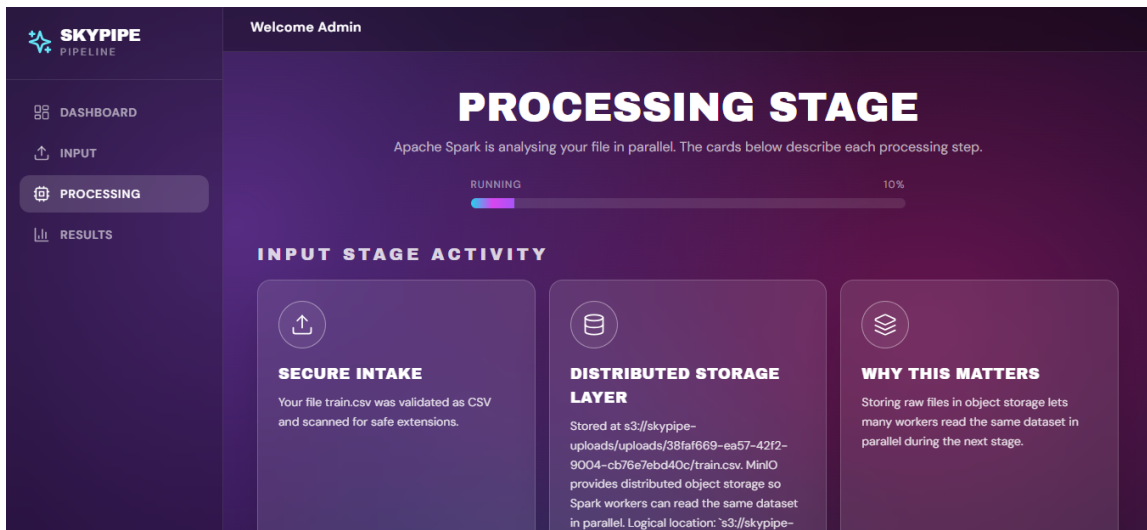


Figure 4.3: Processing page with live progress and step explanations

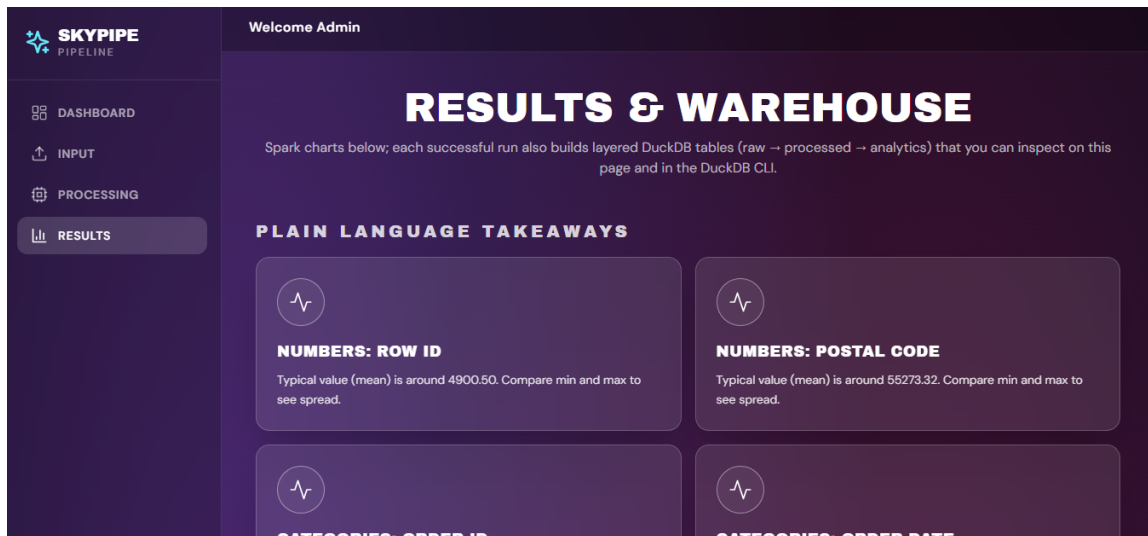


Figure 4.4: Results page with charts and summary cards



Figure 4.5: Results page with charts of different processed data visualizations

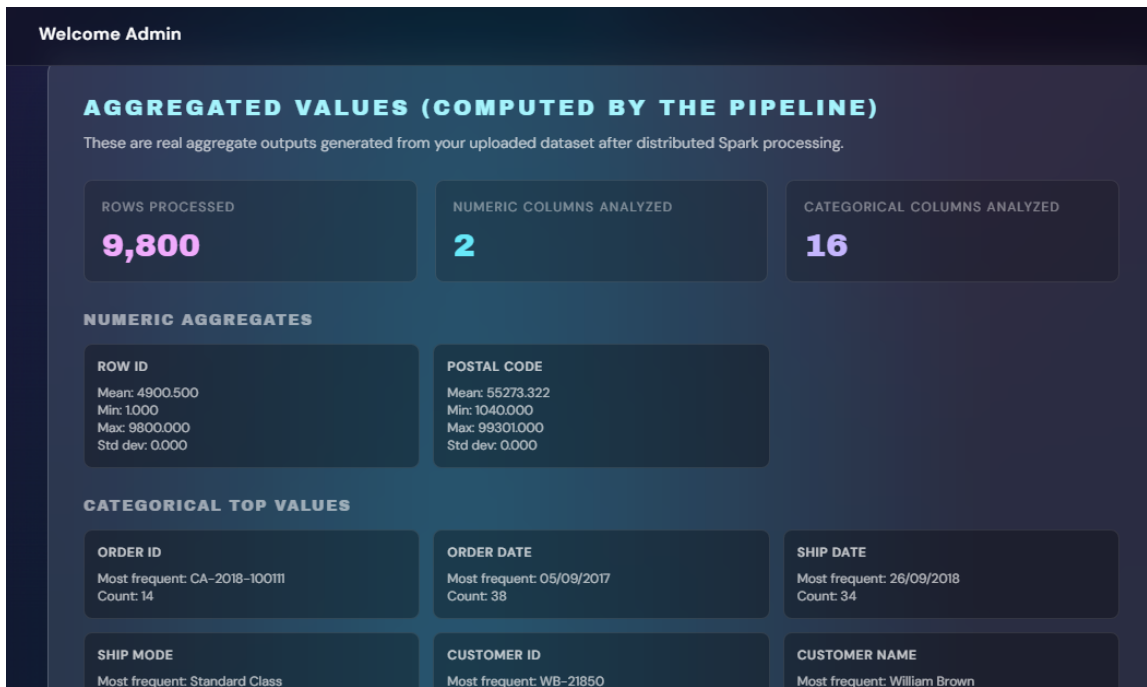


Figure 4.6: Results page with aggregated data from the ingested dataset

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\RUGOGAMU NOELA>duckdb "c:\Users\RUGOGAMU NOELA\Desktop\CLOUD_EXAM_PROJECT\skypipe\data\warehouse.duckdb"
DuckDB v1.5.1 (Variegata)
Enter ".help" for usage hints.
warehouse D SHOW SCHEMAS;
```

database_name varchar	schema_name varchar	current boolean
warehouse	analytics	false
warehouse	main	true
warehouse	processed	false
warehouse	raw_data	false

```
warehouse D SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema IN ('main','raw_data','processed','analytics')
ORDER BY table_schema, table_name;
```

table_schema varchar	table_name varchar
analytics	analytics_sales_8beb6e8aee8a
analytics	analytics_train_35c715f40273
analytics	analytics_train_38faf669ea57
analytics	analytics_train_83266f5c666d
analytics	dim_sales_8beb6e8aee8a
analytics	dim_train_35c715f40273
analytics	dim_train_38faf669ea57
analytics	dim_train_83266f5c666d
analytics	fact_sales_8beb6e8aee8a
analytics	fact_train_35c715f40273
analytics	fact_train_38faf669ea57

Figure 4.7: DuckDB showing the layered warehouse tables

Chapter 5

Asset Inventory

SkyPipe consists of several key components and processing artifacts:

Asset	Category	Details
React Frontend	User Interface	Clean multi page web application for upload, monitoring and results.
Nginx	Web Gateway	Serves the frontend and securely routes API requests.
FastAPI Service	Backend API	Handles authentication, uploads and job management.
MinIO	Distributed Storage	S3 compatible object storage for uploaded datasets.
PySpark Runtime	Data Processing	Parallel analytics engine with automatic schema detection.
DuckDB	Data Warehouse	Persistent layered storage for raw, processed and analytics data.
Docker Compose	Deployment	Makes the entire system easy to run locally or in the cloud.

The analytics artifacts (numeric summaries, categorical profiles and datetime trends) are generated dynamically from each dataset and stored as warehouse tables.

Chapter 6

Scalability Analysis

SkyPipe is built to grow easily with increasing data volume. Object storage separates uploads from processing so that many workers can read the same files at the same time. Spark runs calculations in parallel across partitions allowing larger files to be handled simply by adding more processing resources. The web, API and storage services can each be scaled independently. Results saved in the DuckDB warehouse avoid repeating heavy calculations for later use. In testing, medium sized files processed quickly on a laptop. For very large datasets, the system can switch to cloud storage and run Spark on a cluster with adjusted memory and partitioning settings.

Chapter 7

Fault Tolerance Mechanisms

Several features keep the system reliable. Every upload gets a unique job ID that prevents conflicts. If a problem occurs, the job is clearly marked as failed and the user is notified. Processing runs in the background so a single failure does not crash the website. All final results are safely saved in the DuckDB warehouse, so data is never lost even if the browser is closed. Spark's built in retry logic handles temporary issues and Docker containers can be restarted individually without losing stored files.

Chapter 8

Security Configuration

Security is built into every layer. Users must log in with secure JWT tokens before they can upload or view data. File uploads are checked for allowed formats and size limits to prevent misuse. Sensitive settings are stored in environment variables instead of code. Nginx acts as a secure front door, controlling all incoming requests. Storage access is limited to specific buckets and folders. These measures protect the system while keeping it simple to use.

Chapter 9

Technical Justification of Tools and Settings

9.1 Why FastAPI

FastAPI was chosen because it is fast, easy to develop with and produces clear, well documented APIs.

9.2 Why Spark

Spark provides true parallel processing and works well with many file types exactly what is needed for distributed data handling.

9.3 Why MinIO

MinIO gives the benefits of distributed object storage without needing a full cloud account, making the system easy to test and deploy.

9.4 Why DuckDB

DuckDB is lightweight, extremely fast for analytics and stores everything in a single file that is simple to back up and inspect.

9.5 Why Nginx and React

Nginx provides a professional web gateway while React creates a smooth, modern user interface that clearly shows every stage of the pipeline.

Chapter 10

Conclusion and Learning Outcomes

SkyPipe successfully delivers a practical distributed data processing pipeline that is flexible, secure and easy to use. The system handles the full journey from file upload to warehouse storage while remaining transparent at every step.

During development we faced challenges such as managing long processing times without freezing the website and ensuring the warehouse layers were created correctly for every job. These were solved by using automation, careful error handling and consistent initialisation scripts.

The project strengthened our understanding of distributed storage, parallel processing, service architecture and data warehousing. Most importantly, it showed how good design choices make complex systems both powerful and approachable. SkyPipe is now ready for real world use and can be extended as data needs grow.

hyperref

.1 Github Project Link

The complete source code for the cloud computing project is available on GitHub:

https://github.com/RUGOGAMUNOELA/cloud_computing_project.git

Bibliography

- [1] Apache Spark Foundation, “Apache Spark Documentation,” [Online]. Available: <https://spark.apache.org/docs/latest/>.
- [2] DuckDB Labs, “DuckDB Documentation,” [Online]. Available: <https://duckdb.org/docs/>.
- [3] S. Ramírez, “FastAPI Documentation,” [Online]. Available: <https://fastapi.tiangolo.com/>.
- [4] Nginx Inc., “Nginx Documentation,” [Online]. Available: <https://nginx.org/en/docs/>.
- [5] MinIO Inc., “MinIO Documentation,” [Online]. Available: <https://min.io/docs/>.
- [6] Docker Inc., “Docker Documentation,” [Online]. Available: <https://docs.docker.com/>.